Phase 1: Data Exploration & Preprocessing Report

1. Introduction & Objectives

This report details the outcomes of Phase 1 of the dataset preparation and Retrieval-Augmented Generation (RAG) pipeline setup. The primary objectives included auditing the COCO dataset, establishing rigorous data splits, conducting exploratory data analysis, and setting up the initial vector search architecture.

2. Data Exploration & Statistical Analysis

Object Category Distribution

An initial audit of the COCO dataset was conducted to understand the distribution of objects. The dataset exhibits a long-tail distribution where a few categories dominate the annotations.

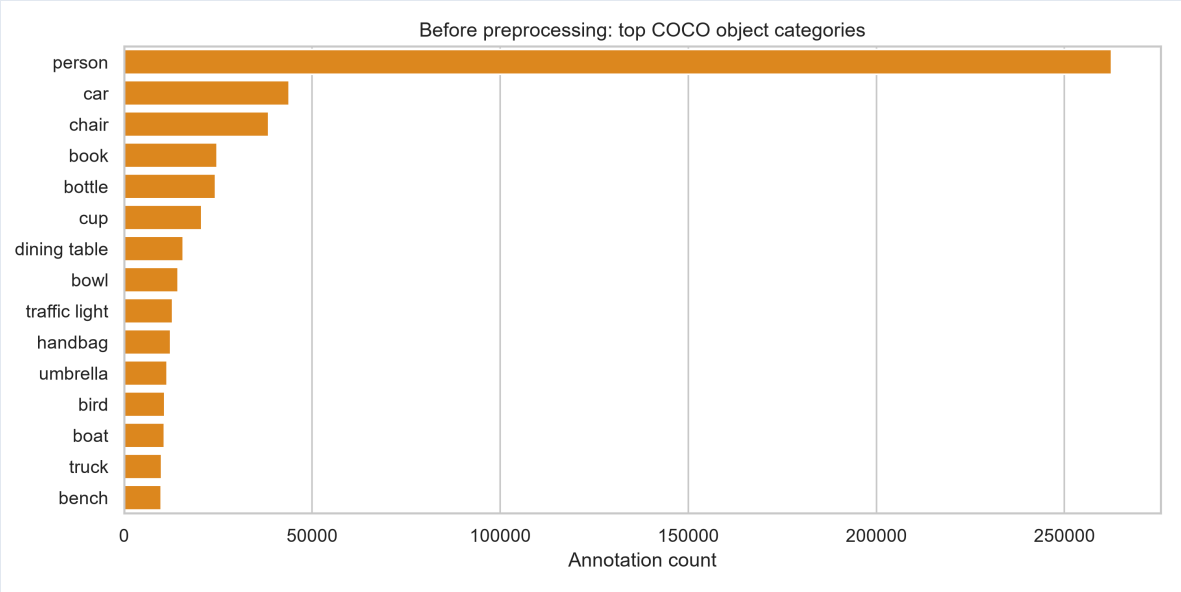


Figure 1: Top COCO Categories Prior to Processing

The most frequently annotated objects are "person," "car," and "chair." Extending this analysis to the top 50 categories confirms that everyday items and vehicles constitute the bulk of the visual data across the dataset.

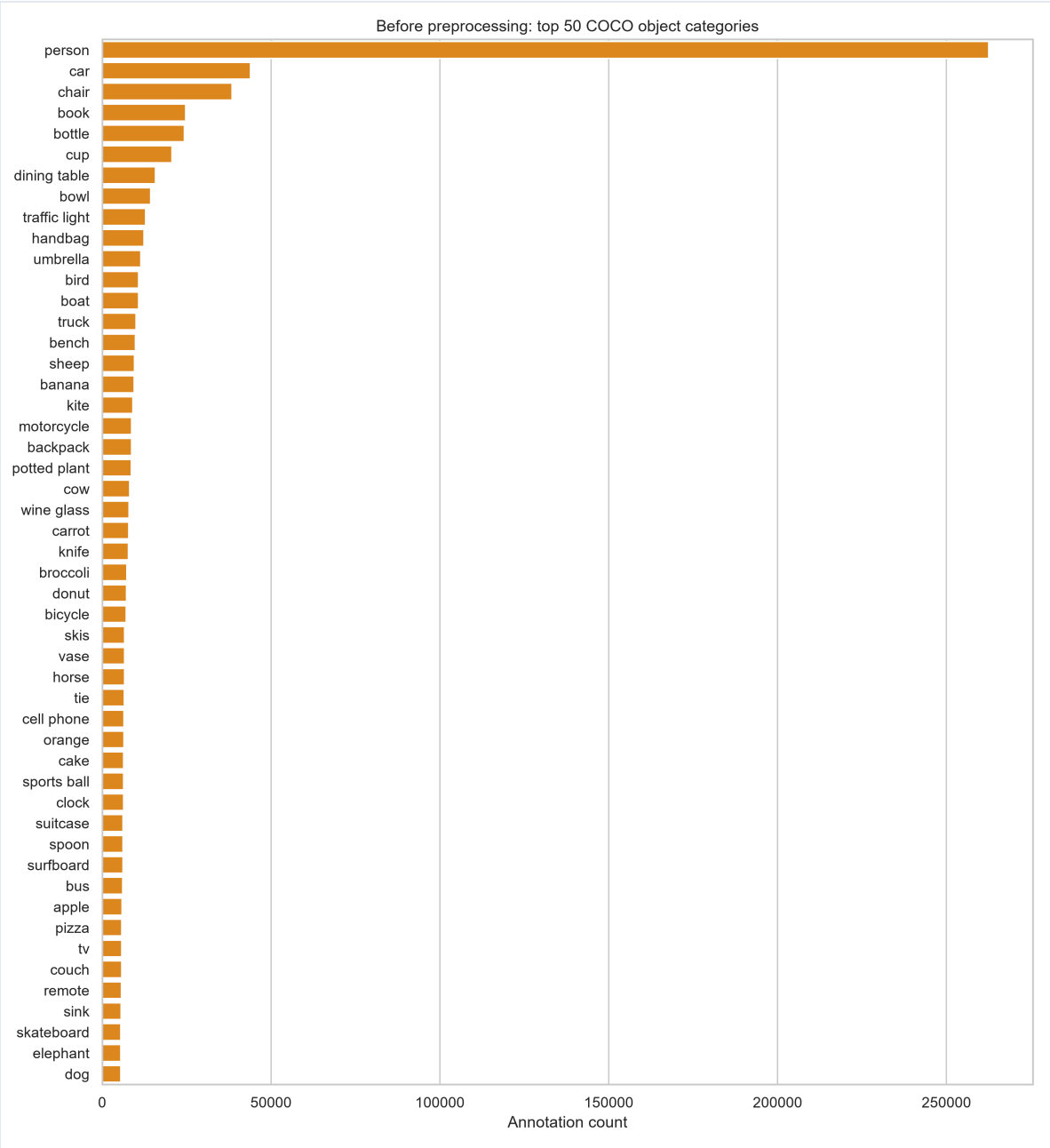


Figure 2: Top 50 COCO Object Categories

Caption Lengths & Scene Complexity

Analysis of the human-written COCO captions reveals a consistent structure. The vast majority of descriptions range between 8 and 15 words, indicating a standard level of descriptive detail provided by human annotators.

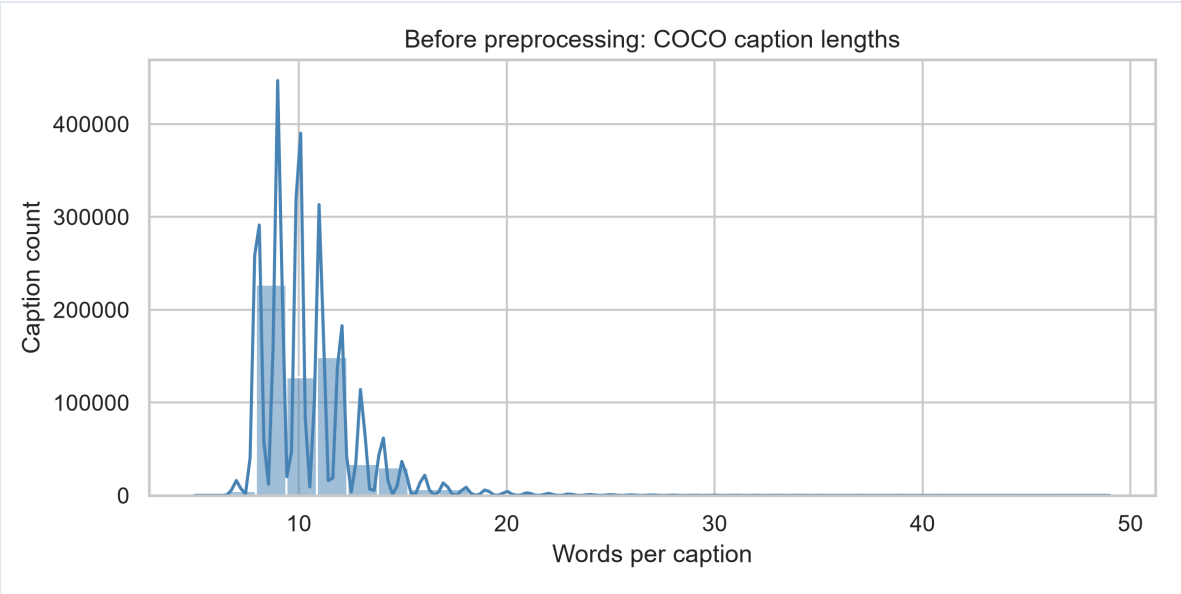


Figure 3: Distribution of Caption Lengths

When examining scene complexity, defined by the number of annotated objects per image, the data is heavily skewed towards simpler scenes. Most images contain fewer than 10 objects, though there is a long tail of highly complex scenes containing over 40 distinct object annotations.

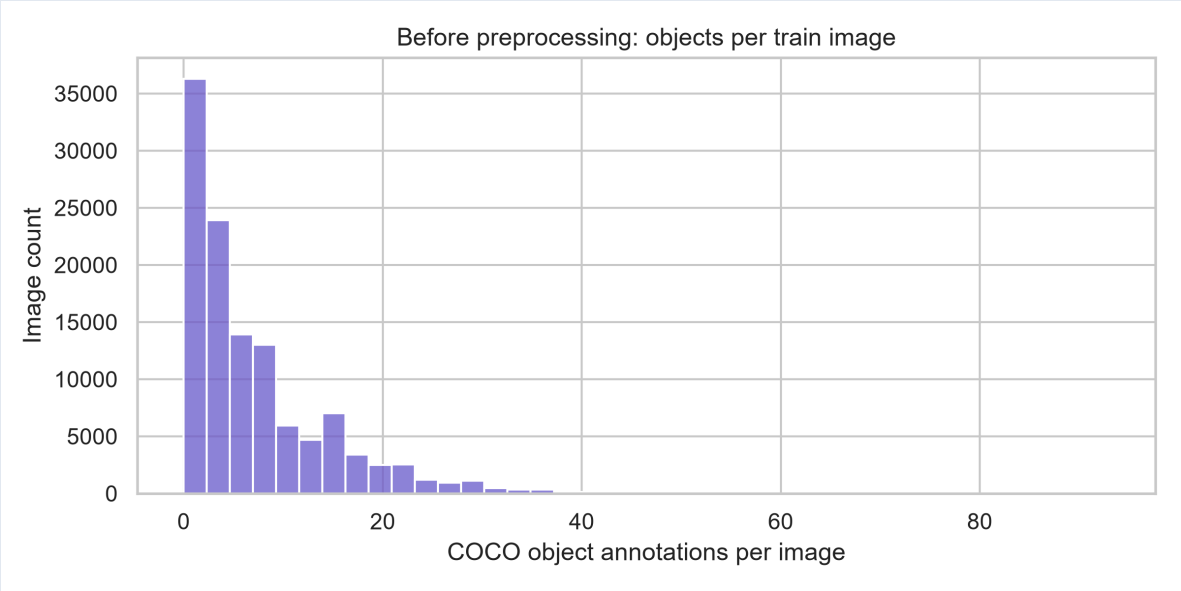


Figure 4: Object Annotation Density per Image

Interestingly, a comparison of scene complexity against average caption length shows no strict correlation. Denser scenes (more object annotations) do not reliably produce longer descriptive captions. Most average caption lengths remain clustered between 8 and 16 words, regardless of the underlying object count.

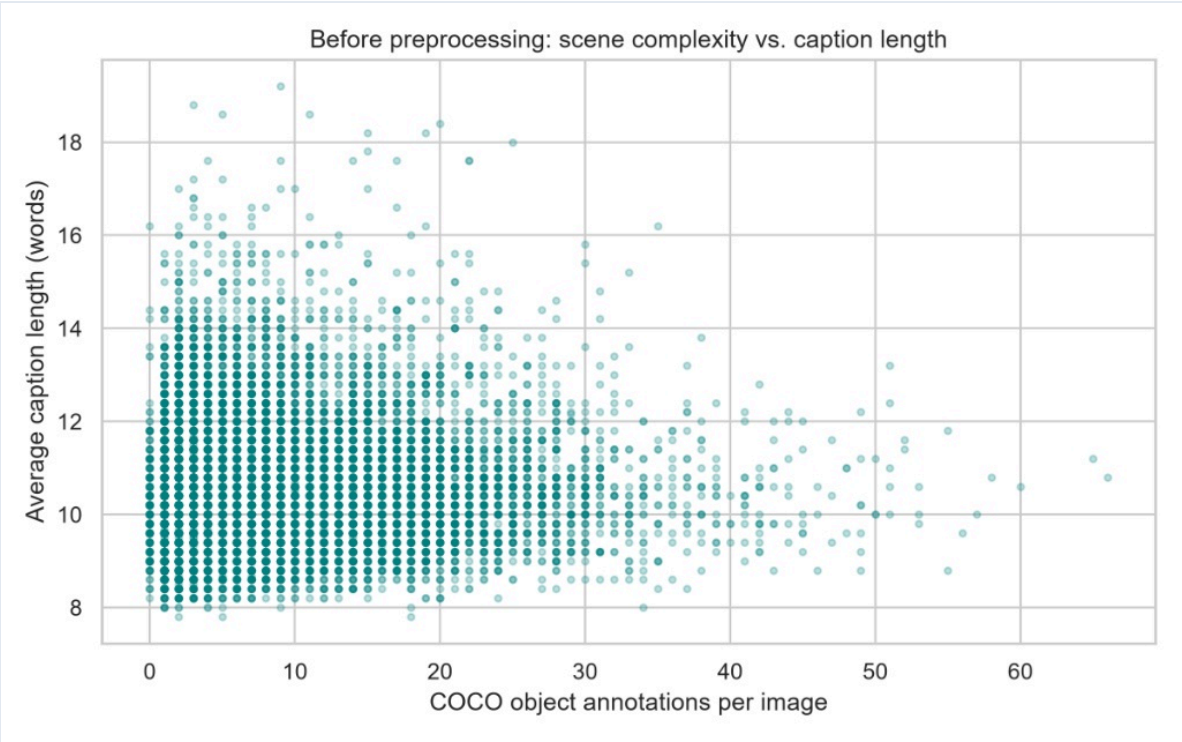


Figure 5: Scene Complexity vs. Caption Length

3. Data Cleaning & Integrity

To ensure dataset integrity, a robust cleaning pipeline was implemented targeting three core areas:

- **Corruption Checks:** Every candidate image was verified using Pillow (`Image.verify()`) to catch and remove corrupted files before ingestion.
- **Duplicate Removal:** An exact SHA-256 file-content hashing mechanism was utilized to detect and remove duplicate images, ensuring only one unique copy of any image was retained.
- **Missing Annotations:** The pipeline audited the dataset for missing annotation-linked files, guaranteeing that all selected images have the required human-written captions and instance data.

4. Revised Data Splitting Architecture

The dataset splitting logic was carefully configured to fully comply with project requirements:

- **Knowledge Base (Training Set):** 10,000 clean, unique images selected from the `train2017` directory to serve as the core retrieval database.
- **Validation Set:** Exactly 100 images randomly sampled **directly from within** the 10,000-image Knowledge Base. This set evaluates retrieval quality and allows for tuning RAG parameters during development.

- **Test Set:** A strictly disjoint set of exactly 100 images drawn from the `val2017` directory. These images remain unseen by the Knowledge Base and are reserved exclusively for final model testing.

5. Object Extraction & RAG Pipeline Setup

YOLOv8 Inference

The `yolov8n.pt` model was deployed to extract objects from the images using a 0.25 confidence threshold. The top detections align closely with the ground truth dataset distribution, successfully identifying high-frequency items like "person," "car," "chair," "cup," and "bottle."

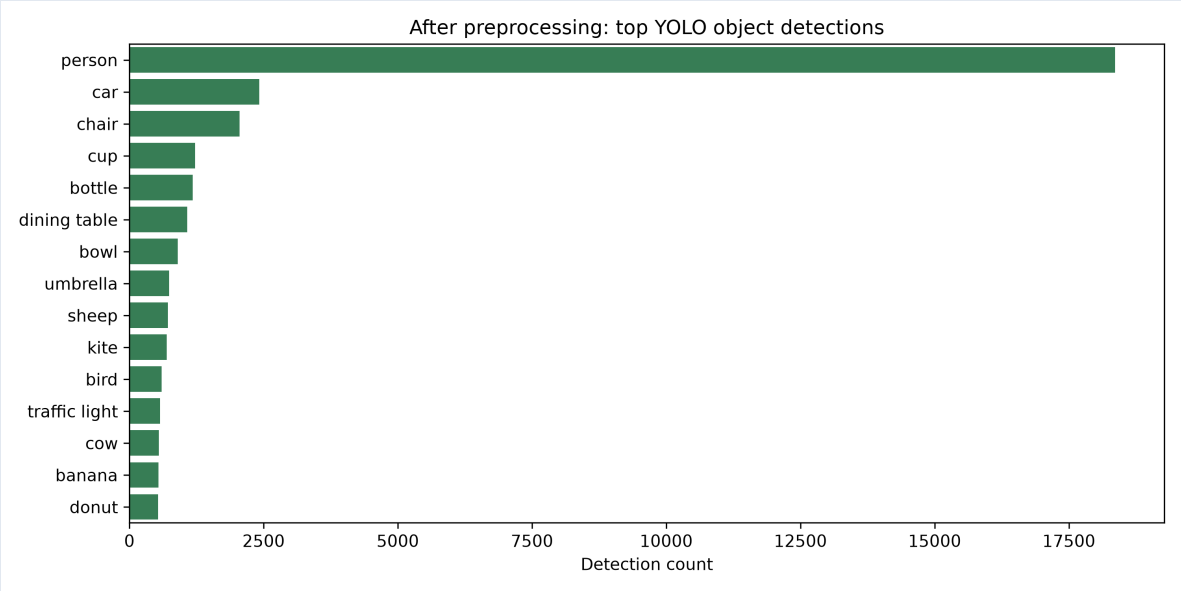


Figure 6: Top Extracted YOLOv8 Detections

Query Reformulation & Vector Indexing

To power the Retrieval-Augmented Generation (RAG) system, the following pipeline was established:

- **Query Reformulation Layer:** Prior to querying the database, a query reformulation step rewrites user prompts using a language model. This expands synonyms and clarifies intents, ensuring the query text better matches the exact vocabulary extracted by the YOLO model.
- **Vector Embeddings:** Extracted object strings (e.g., "person, car, traffic light") are encoded into 384-dimensional dense vectors using the `all-MiniLM-L6-v2` sentence transformer.
- **FAISS Indexing:** The embeddings are indexed using `faiss.IndexFlatIP` (Inner Product/Cosine Similarity) to enable fast and scalable semantic retrieval across the knowledge base.